specifically) to URLs, but those MVC controllers must adhere strictly to the conventions you define. Traditionally, applications using MVC controllers to generate HTML tend to use this approach to routing. The downside of this approach is that customizing URLs for a subset of controllers and actions is tricky.

Alternatively, you can use explicit routing to tie a given URL to a specific endpoint. You've seen this approach with minimal APIs, where each endpoint is directly associated with a route template. You can also use explicit routing with MVC controllers by placing `[Route]` attributes on the action methods themselves, hence explicit-routing is also often called *attribute-routing*.

Explicit routing provides more flexibility than convention-based based routing, as you can explicitly define the route template for every action method. Explicit routing is generally more verbose than the convention-based approach, as it requires applying attributes to *every* action method in your application. Despite this, the extra flexibility can be useful, especially when building APIs.

Somewhat confusingly, Razor Pages use *conventions* to generate *explicit routes*! In many ways this combination gives you the best of both worlds: the predictability and terseness of convention-based routing with the easy customization of explicit routing. There are tradeoffs to each of the approaches, as shown in table 14.1.

Table 14.1 The advantages and disadvantages of the routing styles available in ASP.NET Core